COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010-CPE21S3 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	7/28/2026	**DATE SUBMITTED:**	8/1/2026
NAME:	Gabriel Immanuel Serrano	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (std::stack).

ILOA > main.cpp > ...

```cpp
#include <iostream>
#include <stack> // call the stack form the STL

int main(){
    std::stack<int> newStack; // created an object
    std::stack<int> newStack2;

    // testing the empty() function to check if the stack is empty
    std::cout <<"Is the stack empty? " << newStack.empty() << std::endl;

    // insert a 3 elements:
    newStack.push(1);
    std::cout <<"Top of stack: " << newStack.top() << std::endl; // returns whatever on top of the stack
    newStack.push(2);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;
    newStack.push(3);
    std::cout <<"Top of stack: " << newStack.top() << std::endl;

    newStack2.push(4);
    std::cout <<"Top of stack: " << newStack2.top() << std::endl; // returns whatever on top of the stack
    newStack2.push(5);
    std::cout <<"Top of stack: " << newStack2.top() << std::endl;
    newStack2.push(6);
    std::cout <<"Top of stack: " << newStack2.top() << std::endl;

    std::cout <<"Is the stack empty? " << newStack.empty() << std::endl;

    std::cout <<"Size of the stack is: " << newStack.size() << std::endl; // check the size of the stack

    // swap?? -> swap the content of the public member functions
    newStack.swap(newStack2);

    //print the stack 1
    std::cout<<"newStack = ";
    while (!newStack.empty()){
        std::cout<<newStack.top()<< " ";
        newStack.pop();
    }
```

```
39
40      std::cout << "" << std::endl;
41
42      //print the stack 2
43      std::cout<<"newStack2 = ";
44      while (!newStack2.empty()){
45          std::cout<<newStack2.top()<< " ";
46          newStack2.pop();
47      }
48
49      std::cout << "" << std::endl;
50      // emplace?? -> construct the insert element -> public member function
51      newStack.emplace(8);
52      newStack2.emplace(9);
53
54      std::cout << "newStack = ";
55          while (!newStack.empty()) {
56      std::cout << newStack.top() << " ";
57      newStack.pop();
58      }
59
60      std::cout << "newStack2 = ";
61          while (!newStack2.empty()) {
62      std::cout << newStack2.top() << " ";
63      newStack2.pop();
64      }
65      return 0;
66  }
```

Output:

```
Is the stack empty? 1
Top of stack: 1
Top of stack: 2
Top of stack: 3
Top of stack: 4
Top of stack: 5
Top of stack: 6
Is the stack empty? 0
Size of the stack is: 3
newStack = 6 5 4
newStack2 = 3 2 1
newStack = 8 newStack2 = 9
```

For ILO A, we created two objects named newStack and newStack2 using the Stack library. After creating the object, we check whether each stack was empty by using the empty() function. This function returns true if the stack contains no elements. Next, we inserted three elements into newStack and newStack2 using the push() function. This function adds a new element to the top of the stack. After inserting the elements, we used the top() function to display the value at the top of the stack. We then called empty() again to check if the stack is no longer empty. Now the stack contains 3 elements and we used the size() function to check if the size for each object was 3. After that we tested the swap function, swap the value of newStack and newStack2. For the printing we used a while loop, this while loop prints the top element then uses the pop() function until we print all the values. Finally, we used the emplace() function to insert new elements into the stack. The stack now only contains 8 and 9 because we popped all the elements before the emplace function.

Code Analysis

ILO B.1: Implement a stack using a Static Array.

ILOB > main.cpp > ...

```cpp
#include <iostream>

const size_t maxCap = 100;
int stack[maxCap];
int top = -1, newData, elements;

//prototyping
bool isEmpty();
bool isFull();
void push();
void pop();
void Top();
void displayAll();

int main(){
    int choice;
    //asked the user for the maximum elements to be inserted
    std::cout<<"Enter the number of maximm elements for the new stack (max: 100): ";
    std::cin >> elements;

    while(true){
        std::cout<<"Stack Operations:" << std::endl;
        std::cout<<"1.PUSH \n2.POP \n3.TOP \n4.isEmpty \n5.isFull \n6.displayAll" << std::endl;
        std::cin >> choice;

        switch(choice){
            case 1: push();
            break;
            case 2: pop();
            break;
            case 3: Top();
            break;
            case 4: std::cout << isEmpty() << std::endl;
            break;
            case 5: std::cout << isFull() << std::endl;
            break;
            case 6: displayAll();
            break;
            default: std::cout <<"Invalid Choice" << std::endl;
            break;
        }
    }
```

```cpp

    return 0;
}

bool isEmpty(){
    //top is negative -1
    if(top == -1) return true;
    return false;
}
```

```
52
53  bool isFull(){
54      // not equal to the maxCap
55      if(top >= elements){
56          std::cout<< "Stack Overflow" << std::endl;
57          return true;
58      }
59      return false;
60  }
61
62  void push(){
63
64      //check if the stack is full, if yes return an error
65      if(isFull()) return;
66      //ask the user for a value to be pushed into the stack
67      std::cout<<"What number you want to add to the stack? ";
68      std::cin >> newData;
69      //store inside the stack??
70      stack[++top] = newData;
71  }
72
73  void pop(){
74      //check if there are no more elements inside the stack
75      if(isEmpty()) return;
76      //display the elemen to be popped:1
77      std::cout <<"Popping: "<< stack[top] << std::endl;
78      //decrement the top
79      top--;
80  }
81
82  void Top(){
83      //error checking?
84      if(isEmpty()) return;
85
86      //display the top element
87      std::cout << "The top element is: " << stack[top]<< std::endl;
88  }
```

```
89
90  void displayAll(){
91      //error checking
92      if(isEmpty()) return;
93
94      std::cout << "Printing all the values from the stack" << std::endl;
95
96      for(int i = top; i >= 0; i--){
97          std::cout << stack[i] << std::endl;
98      }
99  }
100
```

Output:

```
Enter the number of maximm elements for the new stack (max: 100): 2
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
1
What number you want to add to the stack? 9
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
1
What number you want to add to the stack? 8
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
6
Printing all the values from the stack
8
9
```

```
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
2
Popping: 8
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
6
Printing all the values from the stack
9
```

```
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
2
Popping: 9
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
4
1
```

```
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
5
0
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
```

```
What number you want to add to the stack? 8
Stack Operations:
1.PUSH
2.POP
3.TOP
4.isEmpty
5.isFull
6.displayAll
3
The top element is: 8
```

For the ILO B, we implemented a stack using an array with a maximum capacity of 100 elements. The program asks the user to enter the maximum number of elements allowed in the stack, then it shows the menu by using switch case. This menu is where the user can use each operation like push, pop, top, isEmpty, isFull, and displayAll. The push() function adds a new value to the top of the stack by incrementing the stack top which then would equal to the newData that the user input. The pop() removes and displays the current top value by decrementing the top. The Top() function shows the value at the top by calling the last data on the stack, and the displayAll() prints all the stored values from top to bottom. All of this code has a catch function of isEmpty() function, the isEmpty function checks whether the top is equal to -1, if the condition is true it will return 1. Lastly the isFull() checks whether the stack has reached the maximum number allowed which is 100 , it checks it by decrementing by 1 the elements.

Code analysis

ILOB 2
Header

ILOB2.h > ...

```
#ifndef ILOB2_H
#define ILOB2_H
#include <string>


//class node with data and next members
template <typename T>
class Node{
    public:
    T data;
    Node<T> *next = nullptr;
};


//create a pointer/reference tip that always points to curren top node


//global
template <typename T>
Node<T>* head = nullptr;


//pushing operation
template <typename T>
void push(T newData){
    //create a new node
    Node<T>* newNode = new Node<T>;


    //assign the data to the newnode
    newNode->data = newData;
```

```
32
33
34      //pointer the newNode next to the current top
35      newNode->next = head<T>;
36
37
38      //assign the new Top to the newNode
39      head<T> = newNode;
40
41
42  }
43  //isEmpty Operation
44  template <typename T>
45  bool isEmpty(){
46      return head<T> == nullptr;
47  }
48
49
50  template <typename T>
51  T pop(){
52      //if the stack is empty error catch
53      if(isEmpty<T>()){
54          std::cout<<"Stack Underflow"<< std::endl;
55      }
56      //saving a current top node to a temporary variable
57      Node<T> *temp = head<T>;
58      //store the data to a temporary variable
59      T tempVal = temp->data;
60      //move the head to next node
61      head<T> = head<T>->next;
62      //free up the memory
63      delete temp;
64      //return the temporary variable
65      return tempVal;
66  }
```

```
67
68
69  template<typename T>
70  void Top(){
71      //what if the stack is empty
72      if(isEmpty<T>()) return;
73
74
75      std::cout<<"The Top is: " << head<T>->data;
76
77
78  }
79
80  template <typename T>
81  void displayAll(){
82      if(isEmpty<T>()) return;
83
84
85      while(head<T> != nullptr){
86          std::cout << head<T>->data <<" ";
87          head<T> = head<T>->next;
88      }
89
90
91  }
92  #endif
```

Main.py

```cpp
#include <iostream>
#include "ILOB2.h"

int main(){
    std::cout<<"------ Pushing 'I 'E' 'E' 'E' ------ \n";
    //pushing
    push('I');
    push('E');
    push('E');
    push('E');

    //display top
    Top<char>();

    //display all elements
    std::cout<<"\n------ DISPLAY ALL ------ \n";
    displayAll<char>();

    //pop and print each element
    std::cout<<"\n------ Popping 'I 'E' 'E' 'E' ------ \n";
    while(!isEmpty<char>()){
        std::cout<<"popping: "<<pop<char>()<<std::endl;
    }

    //test stack overflow
    pop<char>();

    return 0;
}
```

OUTPUT :

```
------ Pushing 'I 'E' 'E' 'E' ------
The Top is: E
------ DISPLAY ALL ------
E E E I
------ Popping 'I 'E' 'E' 'E' ------
Stack Underflow
```

For the ILO B, we implemented a stack using a linked list instead of an array. The program first pushes the characters I, E, E, and E into the stack by calling the push() function. Each time a new value is inserted, a new node is created and placed at the top of the stack by updating the head pointer. After pushing the elements, the Top() function displays the current top element without removing it. The displayAll() function then prints all the elements from the top of the stack down to the last node. After displaying the stack, the program calls the pop() function that is inside a while loop, it will run and run until the stack is empty. It works by removing the top node and updating the head pointer to the next node. The isEmpty() function is used to check whether the stack contains any node by checking the head pointer if it's equal to null. Lastly the program calls the pop() once more to see if the stack is underflow, which would print a message saying "Stack Underflow".

Code analysis

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

a. Stack using Arrays

supplementaryA.cpp > ...

```cpp
#include <iostream>
#include <string>

const int maxCap = 100;
char stack[maxCap];
int top = -1;

bool isEmpty();
void push(char newData);
char pop();
bool isOpenClose(char opening, char closing);

int main(){
    std::string opening = "([{";
    std::string closing = ")]}";
    std::string choice;
    bool balanced = true;

    std::cout << "Balanced Symbol Checker (Array Stack)" << std::endl;
    std::cout << "Enter Expression: ";
    std::cin >> choice;

    for (int i = 0; i < choice.length(); i++) {

    if (opening.find(choice[i]) != std::string::npos) {
        push(choice[i]);
    }
    else if (closing.find(choice[i]) != std::string::npos) {

        if (isEmpty()) {
            balanced = false;
            break;
        }

```

```
            if (!isOpenClose(pop(), choice[i])) {
                balanced = false;
                break;
                }
            }
        }

        if (!isEmpty()) {
            balanced = false;
        }

        if (balanced)
            std::cout << "Balanced Expression";
        else
            std::cout << "Unbalanced Expression";

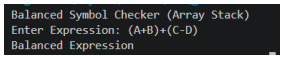
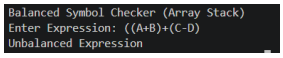
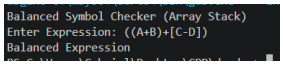
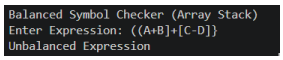
        return 0;
    }

    bool isEmpty(){
        return top == -1;
    }

    void push(char newData){
        top++;
        stack[top] = newData;
    }

    char pop(){
        char newData = stack[top];
        top--;
        return newData;
    }

    bool isOpenClose(char opening, char closing) {
        if (opening == '(' && closing == ')') return true;
        if (opening == '{' && closing == '}') return true;
        if (opening == '[' && closing == ']') return true;
        return false;
    }
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The expression is balanced because every opening parenthesis has a matching closing parenthesis in the correct order.
((A+B)+(C-D)	N		The expression is unbalanced because one opening parenthesis does not have a matching closing parenthesis.
((A+B)+[C-D])	Y		The expression is balanced because all parentheses and brackets are properly matched and correctly nested.
((A+B]+[C-D]}	N		The expression is unbalanced because the closing symbols do not match the starting opening symbols.

b. Stack using Linked List
Header file
** just added this on the end of the ILOB2

```
90
91  bool isOpenClose(char opening, char closing) {
92      if (opening == '(' && closing == ')') return true;
93      if (opening == '{' && closing == '}') return true;
94      if (opening == '[' && closing == ']') return true;
95      return false;
96  }
97  #endif
```

Maincpp

supplementaryB.cpp > ...

```cpp
#include <iostream>
#include <string>
#include "supplementaryb.h"

int main(){
   std::string opening = "([{";
    std::string closing = ")]}";
    std::string choice;
    bool balanced = true;

    std::cout << "Balanced Symbol Checker (Array Stack)" << std::endl;
    std::cout << "Enter Expression: ";
    std::cin >> choice;

    for (int i = 0; i < choice.length(); i++) {

    if (opening.find(choice[i]) != std::string::npos) {
        push(choice[i]);
    }
    else if (closing.find(choice[i]) != std::string::npos) {

        if (isEmpty<char>()) {
            balanced = false;
            break;
        }

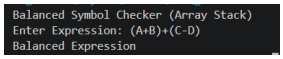
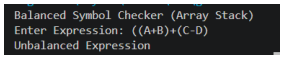
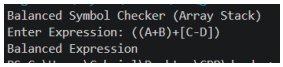
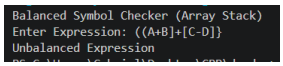
        if (!isOpenClose(pop<char>(), choice[i])) {
            balanced = false;
            break;
            }
        }
    }
```

```cpp

    if (!isEmpty<char>()) {
        balanced = false;
    }

    if (balanced)
        std::cout << "Balanced Expression";
    else
        std::cout << "Unbalanced Expression";

    return 0;
}
```

Expression	Valid? (Y/N)	Output (Console Screenshot)	Analysis
(A+B)+(C-D)	Y		The expression is balanced because every opening parenthesis has a matching closing parenthesis in the correct order.
((A+B)+(C-D)	N		The expression is unbalanced because one opening parenthesis does not have a matching closing parenthesis.
((A+B)+[C-D])	Y		The expression is balanced because all parentheses and brackets are properly matched and correctly nested.
((A+B]+[C-D]}	N		The expression is unbalanced because the closing symbols do not match the starting opening symbols.

For the ILO C, we implemented a delimiter symbol checker using a stack with both an array and a linked list implementation. The program works by scanning the expression one character at a time and pushing every opening symbol onto the stack. Whenever a closing symbol is encountered, the program pops the top element from the stack and checks whether it matches the corresponding opening symbol using the isOpenClose() function from the prototype and the headerfile. If a mismatch is found or there are still unmatched opening symbols left in the stack after the entire character has been popped, the program will display “Unbalanced Expression “ , and if not it will display “Balance Expression”

Code analysis

Tools Analysis:
- To simplify an array based stack you need to have a definite memory size, while the linked list based stack will have a dynamically sized memory size based on your memory, and it will define its own size.

I learned how to implement the stack data structure using both arrays and linked lists in C++. Through this activity, I become familiar with stack operations and how to implement them by prototyping things like push, pop, top and isEmpty. Also the base concept of stack which is LIFO or Last in, First Out. Now moving on to the supplementary , this was a challenge to me because It was very hard to implement on how to properly only push the opening while not pushing the closing, I got the concept of how it works but I can't express it in coding terms. This was one of the most challenging parts in the stack because It required a lot of statements and logics. Overall, this activity was such a fun activity on most of the part but not the supplementary because it mentally and physically challenged me, making me search all over the internet just to understand how it works. I believe that I still need a lot in terms of coding logic but when it comes to understanding how it is supposed to work I am confident that I did a great job on the supplementary.

Conclusion

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ChatGPT and Code Forums like StackOverflow**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar Correction and Code Assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **ChatGPT helped me fix my wordings on the analysis and conclusion, while the stack overflow helped me start on how to do the supplementary, but ChatGPT helped me assisting in the coding, but I didn't necessarily rely on ChatGPT because I still understood how the concept works and how it functions properly.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.